Department of Computer Science & Engineering

Microprocessor & Computer Architecture
UE24CS251B

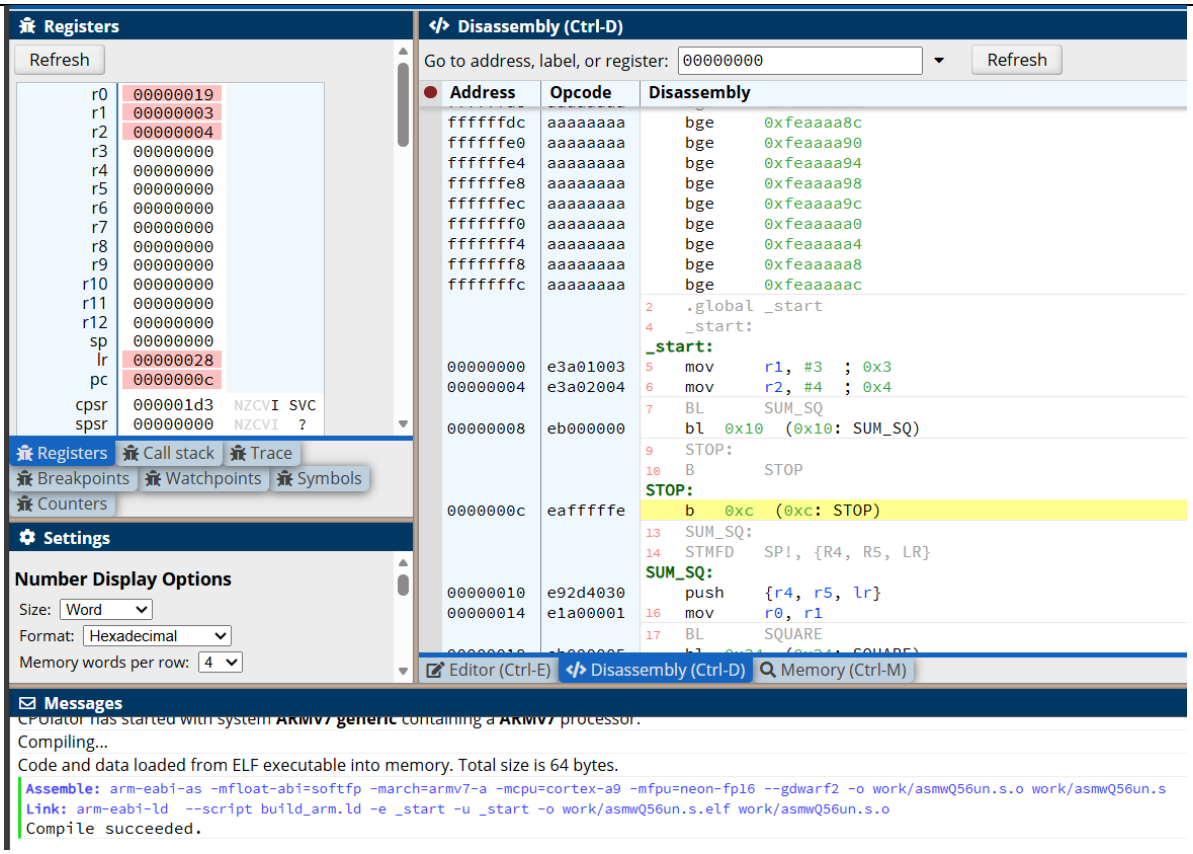
MPCA Lab Programs - 6

Name: DHRITI KAMANI	SRN: PES1UG24AM085	Section: B

1 **Write an ARM7TDMI assembly program using nested procedure calls to compute the sum of squares of two numbers. The main program calls a procedure SUM_SQ, which in turn calls another procedure SQUARE to compute the square of a number.**

Program and Results Screenshot:

```
        .global _start
_start:
        MOV     R1, #3
        MOV     R2, #4
        BL      SUM_SQ
STOP:
        B       STOP
SUM_SQ:
        STMFD   SP!, {R4, R5, LR}
        MOV     R0, R1
        BL      SQUARE
        MOV     R4, R0
        MOV     R0, R2
        BL      SQUARE
        MOV     R5, R0
        ADD     R0, R4, R5
        LDMFD   SP!, {R4, R5, PC}
SQUARE:
        STMFD   SP!, {LR}
        MUL     R0, R0, R0
        LDMFD   SP!, {PC}
```

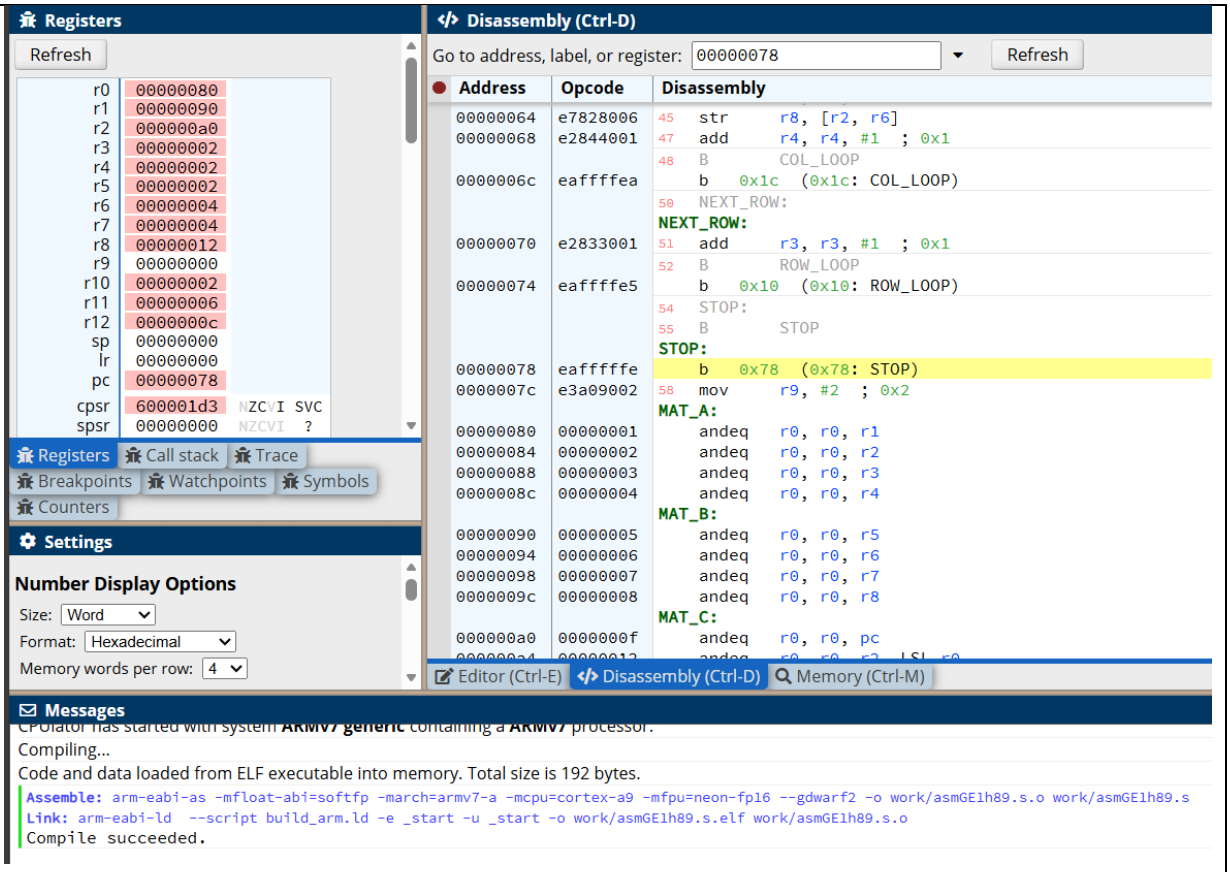


2 **Write an ARM7TDMI assembly language program to multiply two matrices of suitable order and store the result in a separate memory location.**

Program and Results Screenshot:

```
        .global _start
_start:
        LDR     R0, =MAT_A
        LDR     R1, =MAT_B
        LDR     R2, =MAT_C
        MOV     R3, #0
ROW_LOOP:
        CMP     R3, #2
        BEQ     STOP
        MOV     R4, #0
COL_LOOP:
        CMP     R4, #2
        BEQ     NEXT_ROW
        MOV     R5, #0
        MOV     R8, #0
```

```
K_LOOP:
        CMP     R5, #2
        BEQ     STORE
        MLA     R6, R3, R9, R5
        LSL     R6, R6, #2
        LDR     R10, [R0, R6]
        MLA     R7, R5, R9, R4
        LSL     R7, R7, #2
        LDR     R11, [R1, R7]
        MUL     R12, R10, R11
        ADD     R8, R8, R12
        ADD     R5, R5, #1
        B       K_LOOP
STORE:
        MLA     R6, R3, R9, R4
        LSL     R6, R6, #2
        STR     R8, [R2, R6]
        ADD     R4, R4, #1
        B       COL_LOOP
NEXT_ROW:
        ADD     R3, R3, #1
        B       ROW_LOOP
STOP:
        B       STOP
        MOV     R9, #2
MAT_A:
        .word   1, 2
        .word   3, 4
MAT_B:
        .word   5, 6
        .word   7, 8
MAT_C:
        .space  16
```

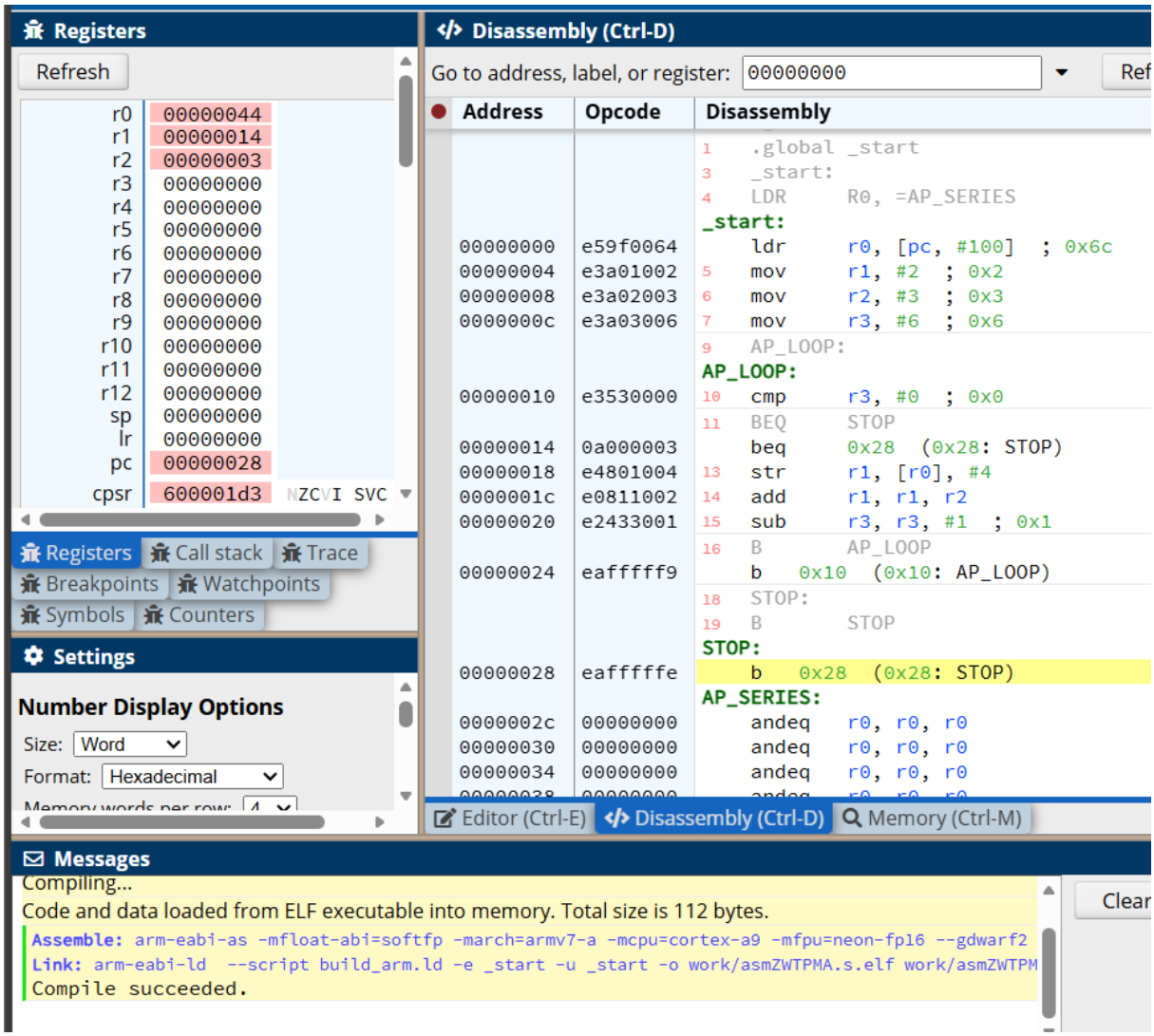


3 **Write an ARM7TDMI assembly language program to generate the first N terms of an arithmetic progression and store them in memory.**

Program and Results Screenshot:

```
        .global _start
_start:
        LDR     R0, =AP_SERIES
        MOV     R1, #2
        MOV     R2, #3
        MOV     R3, #6
AP_LOOP:
        CMP     R3, #0
        BEQ     STOP
        STR     R1, [R0], #4
        ADD     R1, R1, R2
        SUB     R3, R3, #1
        B       AP_LOOP
STOP:
        B       STOP
```

AP_SERIES:
.space 64

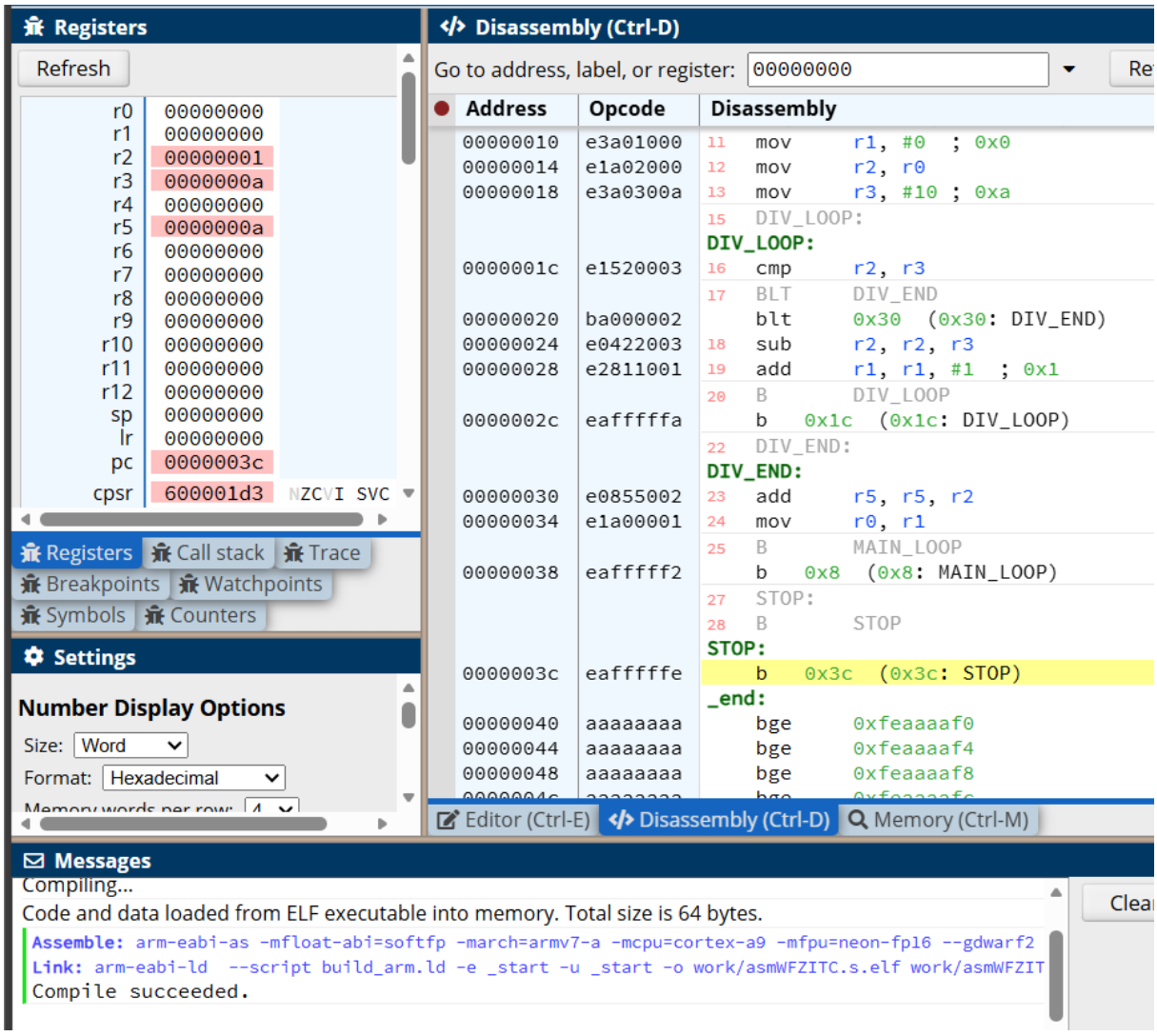


4 **Write an ARM7TDMI assembly language program to compute the sum of digits of a given number using repeated division.**

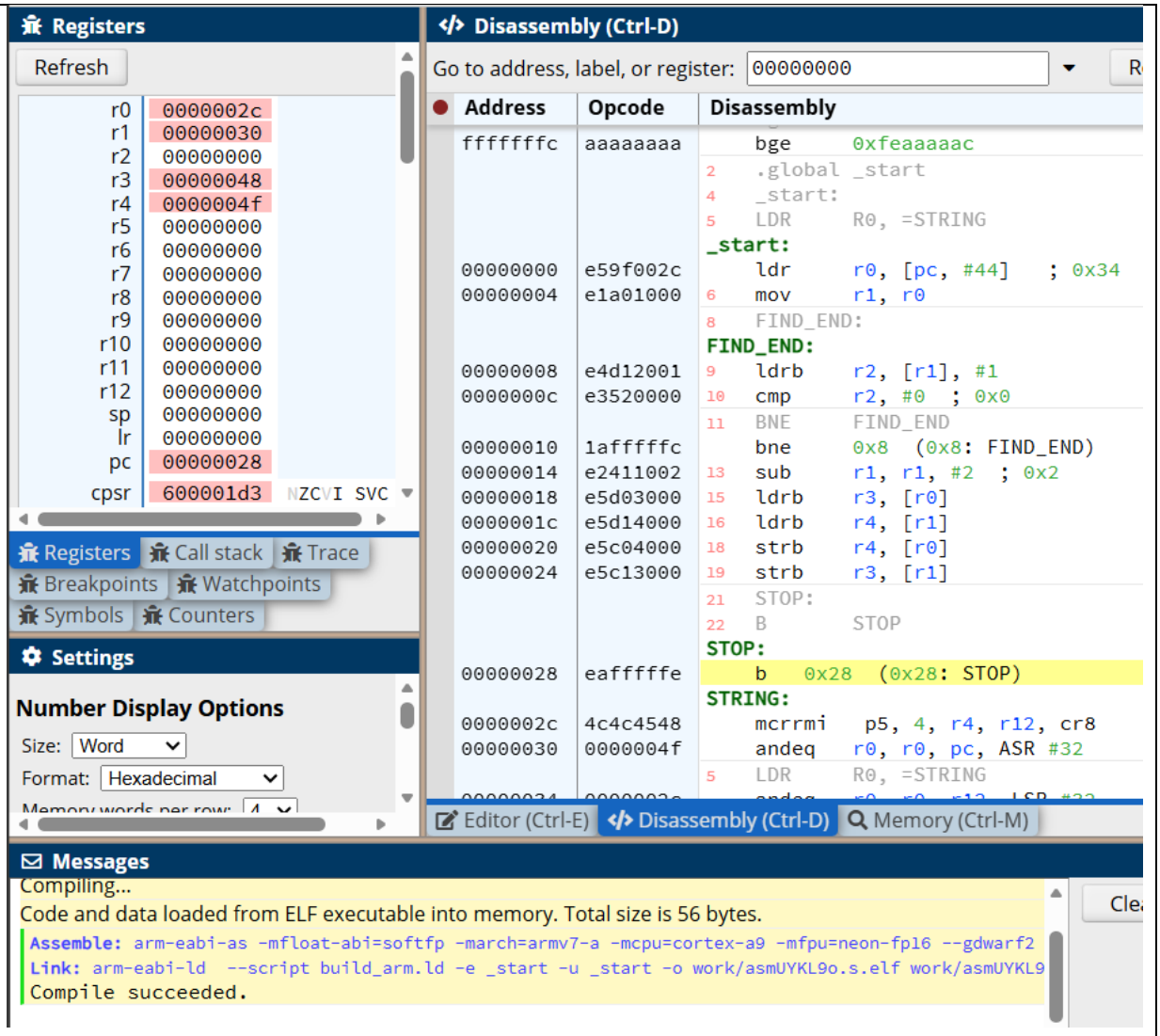
Program and Results Screenshot:

```
        .global _start
_start:
        MOV     R0, #1234
        MOV     R5, #0
MAIN_LOOP:
        CMP     R0, #0
        BEQ     STOP
        MOV     R1, #0
```

```
        MOV     R2, R0
        MOV     R3, #10
DIV_LOOP:
        CMP     R2, R3
        BLT     DIV_END
        SUB     R2, R2, R3
        ADD     R1, R1, #1
        B       DIV_LOOP
DIV_END:
        ADD     R5, R5, R2
        MOV     R0, R1
        B       MAIN_LOOP
STOP:
        B       STOP
```



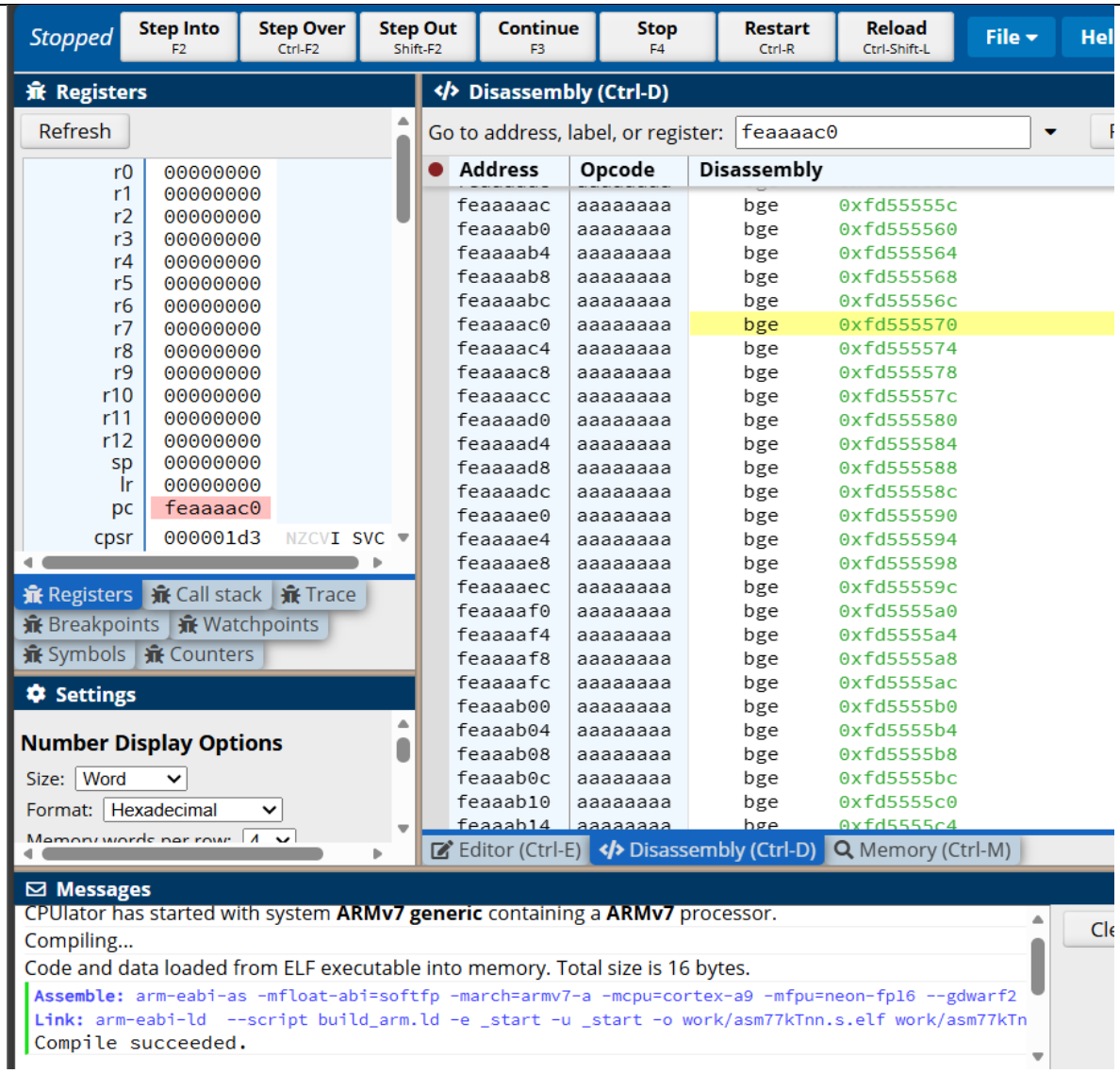
5	**ASSIGNMENT QUESTIONS:**
	
	i) Write a program to swap the first and last character of a given string.
	Program and Results Screenshot:
	.global _start
	_start:
	LDR R0, =STRING
	MOV R1, R0
	FIND_END:
	LDRB R2, [R1], #1
	CMP R2, #0
	BNE FIND_END
	SUB R1, R1, #2
	LDRB R3, [R0]
	LDRB R4, [R1]
	STRB R4, [R0]
	STRB R3, [R1]
	STOP:
	B STOP
	STRING:
	.asciz "HELLO"



ii) Given a c Code convert it in its equivalent Arm Code : z = (a << 2) |(b & 15);

Program and Results Screenshot:

```
LSL   R3, R1, #2
AND   R4, R2, #15
ORR   R0, R3, R4
```



6 **Write an ARM7TDMI assembly language program in which the main program calls a procedure STRING_PROC. The procedure STRING_PROC calls separate procedures LENGTH to find the length of a string and REVERSE to reverse the string. The reversed string must be stored in a separate memory location.**

Program and Results Screenshot:

```
        .global _start
_start:
        LDR     R0, =STRING
        LDR     R1, =REV_STR
        BL      STRING_PROC
STOP:
        B       STOP
```

```
STRING_PROC:
        STMFD   SP!, {R4, R5, LR}
        MOV     R4, R0
        BL      LENGTH
        MOV     R5, R0
        MOV     R0, R4
        MOV     R2, R5
        BL      REVERSE
        LDMFD   SP!, {R4, R5, PC}
LENGTH:
        STMFD   SP!, {R1, LR}
        MOV     R1, #0
LEN_LOOP:
        LDRB    R2, [R0, R1]
        CMP     R2, #0
        BEQ     LEN_DONE
        ADD     R1, R1, #1
        B       LEN_LOOP
LEN_DONE:
        MOV     R0, R1
        LDMFD   SP!, {R1, PC}
REVERSE:
        STMFD   SP!, {R3, R4, R5, LR}
        SUB     R2, R2, #1
        MOV     R3, #0
REV_LOOP:
        CMP     R3, R2
        BGT     REV_DONE
        LDRB    R4, [R0, R2]
        STRB    R4, [R1, R3]
        ADD     R3, R3, #1
        SUB     R2, R2, #1
        B       REV_LOOP
REV_DONE:
        MOV     R4, #0
        STRB    R4, [R1, R3]
        LDMFD   SP!, {R3, R4, R5, PC}
```

```
STRING:
        .asciz  "ARM7TDMI"
REV_STR:
        .space  32
```